



Lab VIVAM

Experiment No. 1

- **Title:** Identify the case study and detail statement of problem. Design an Entity-Relationship (ER) / Extended Entity-Relationship (EER) Model.
- **Summary:** This initial experiment focused on the conceptual database design phase. A case study (likely related to a Parking Management System, based on later experiments) was identified, and the problem requirements were detailed. An ER (or EER) model was designed to visually represent the entities (like Admin, User, Vehicle, Parking Infrastructure), their attributes (like IDs, names, dates, location, capacity), and the relationships between them (like manages, borrows, monitors). The theory covered standard ER/EER notations for entities, attributes, relationships, and constraints (participation, cardinality, specialization/generalization).
- **Code/Implementation:** No SQL code. The implementation involved creating the ER/EER diagram itself (represented by a placeholder in the text).
- **MySQL Syntax Used:** None.

Experiment No. 2

- **Title:** Mapping ER/EER to Relational schema model.
 - **Summary:** This experiment focused on translating the conceptual ER/EER model (created in Exp 1) into a logical relational database schema. The process involves applying specific mapping rules outlined in the theory section: mapping strong entities, weak entities, various relationship types (1:1, 1:N, M:N), and different attribute types (simple, composite, multivalued) into tables with appropriate columns, primary keys, and foreign keys.
 - **Code/Implementation:** No SQL code. The implementation involved defining the structure of the relational tables based on the ER/EER diagram and mapping rules (represented by a placeholder in the text).
 - **MySQL Syntax Used:** None.
-

Experiment No. 3

- **Title:** Create and populate database using Data Definition Language (DDL) and Apply Integrity Constraints for the specified system.
 - **Summary:** This experiment transitioned from design to implementation. Using SQL DDL commands, the database (`pms`) and tables defined in the relational schema (from Exp 2) were created. Key integrity constraints such as `PRIMARY KEY` , `FOREIGN KEY` (often with `ON DELETE CASCADE` for relationship management), and `NOT NULL` were applied to ensure data accuracy and consistency according to the design. Basic DDL operations like describing, renaming, and dropping tables were also demonstrated.
 - **Code/Implementation:** Created a database (`pms`) and multiple tables (`ADMIN` , `USERS` , `PMS` , `PARKING_INFRA` , `VEHICLE` , and several junction/linking tables like `ADMIN_PMS` , `USERS_BORROWS_VEHICLE` , etc.) defining columns with data types and constraints. Showed table structure, renaming, and dropping.
 - **MySQL Syntax Used:** `CREATE DATABASE` , `USE` , `CREATE TABLE` , `INT` , `VARCHAR` , `DATE` , `PRIMARY KEY` , `NOT NULL` , `FOREIGN KEY (...) REFERENCES ... (...)` , `ON DELETE CASCADE` , `DESC` , `RENAME TABLE` , `ALTER TABLE ... RENAME TO ...` , `DROP TABLE` .
-

Experiment No. 4

- **Title:** Apply DML Commands for your specified System.
 - **Summary:** This experiment focused on Data Manipulation Language (DML), which deals with the data *within* the tables. It demonstrated how to insert new records into tables, retrieve existing records, update specific data in records based on conditions, and delete records from tables based on conditions.
 - **Code/Implementation:** Created a sample `employee` table. Used `INSERT INTO` to add rows (including one showing selective column insertion and others inserting all values). Used `SELECT *` to display all data. Used `UPDATE ... SET ... WHERE` to change a department value. Used `DELETE FROM ... WHERE` to remove a specific row.
 - **MySQL Syntax Used:** `CREATE DATABASE` , `USE` , `CREATE TABLE` , `INSERT INTO ... VALUES (...)` , `SELECT * FROM ...` , `UPDATE ... SET ... WHERE ...` , `DELETE FROM ... WHERE ...` . (Also `INT` , `VARCHAR` , `PRIMARY KEY` , `NOT NULL`).
-

Experiment No. 5

- **Title:** Perform Simple queries, string manipulation operations and aggregate functions.
- **Summary:** This experiment focused on data retrieval and summarization. It involved writing simple `SELECT` queries to fetch data and using aggregate functions (`MIN` , `AVG` , `COUNT`) to perform calculations across multiple rows, often grouped by a specific column. Although the title mentions string manipulation, the provided code primarily demonstrates aggregate functions.
- **Code/Implementation:** Created `se1_student` and `dep_se1` tables with a relationship. Inserted data. Performed `UPDATE` and `DELETE` (with a potentially incorrect `id` reference for delete). Used `SELECT MIN(age)` , `SELECT AVG(marks)` , and `SELECT COUNT(*) ... GROUP BY did` to find minimum age, average marks, and count of entries per department ID, respectively.
- **MySQL Syntax Used:** `CREATE TABLE` , `INSERT INTO` , `UPDATE` , `DELETE` , `SELECT MIN(...) AS ... FROM ...` , `SELECT AVG(...) AS ... FROM ...` , `SELECT COUNT(*) FROM ... GROUP BY ...` . (Also `INT` , `VARCHAR` , `DECIMAL` , `PRIMARY KEY` , `NOT NULL` , `FOREIGN KEY`).

Experiment No. 6

- **Title:** Implement SET operators and Datetime functions.
- **Summary:** This experiment explored two key areas: SET operators (`UNION` , `UNION ALL`) for combining the results of multiple `SELECT` statements, and various built-in functions for working with date and time data types.
- **Code/Implementation:** Created `Student` and `Department` tables. Used `UNION` and `UNION ALL` to combine `department_id` values from both tables. Demonstrated datetime functions: `CURRENT_DATE()` to get the current date, date arithmetic (`NOW()` - `INTERVAL 1 YEAR`) for filtering, `DATE_FORMAT()` for custom date string representation, and `EXTRACT()` to get specific parts (year, month) of a date. `INTERSECT` and `EXCEPT` / `MINUS` were noted as potentially unavailable depending on the SQL version/dialect.
- **MySQL Syntax Used:** `CREATE TABLE` , `INSERT INTO` , `SELECT ... UNION SELECT ...` , `SELECT ... UNION ALL SELECT ...` , `SELECT CURRENT_DATE()` , `NOW()` , `INTERVAL` , `DATE_FORMAT()` , `EXTRACT(unit FROM date)` . (Also `INT` , `VARCHAR` , `DATE` , `PRIMARY KEY`).

Experiment No. 7

- **Title:** Nested queries and Complex queries.
- **Summary:** This experiment focused on constructing more sophisticated queries. It involved using joins (`JOIN`) to combine data from multiple related tables based on matching keys and using subqueries (nested `SELECT` statements within the `WHERE` or `SELECT` clause) to filter or retrieve data based on results derived from another query.
- **Code/Implementation:** Created `salesman` and `orders` tables. Wrote queries using `JOIN ... ON ... WHERE` to find orders based on salesman name or city. Wrote a query using a subquery `WHERE purch_amt > (SELECT AVG(purch_amt) ...)` to find orders with purchase amounts greater than the average for a specific date. Used `DISTINCT` with joins.
- **MySQL Syntax Used:** `CREATE TABLE` , `INSERT INTO` , `SELECT ... FROM ... JOIN ... ON ... WHERE ...` , `SELECT ... FROM ... WHERE ... > (SELECT AVG(...) ...)` , `SELECT DISTINCT ... FROM ... JOIN ... ON ...` . (Also `INT` , `VARCHAR` , `DECIMAL` , `DATE` , `PRIMARY KEY` , `NOT NULL` , `FOREIGN KEY`).

Experiment No. 8

- **Title:** Procedures and Functions.
 - **Summary:** This experiment introduced reusable database code blocks: Stored Procedures and Stored Functions. Procedures (`CREATE PROCEDURE`) are blocks of SQL code for performing specific tasks (which can have input/output parameters), while Functions (`CREATE FUNCTION`) are blocks that perform a calculation and *must* return a single value.
 - **Code/Implementation:** Created an `Employees_SE1` table. Created a procedure `GetEmployeesByDept` taking `dept_id` as input (`IN`) to select employees. Created a function `GetTotalSalaryByDept` taking `dept_id` as input, calculating the sum of salaries (`SUM(Salary)`), and returning the result (`RETURNS DECIMAL`). Demonstrated the use of `DELIMITER` for defining these blocks, `CALL` to execute the procedure, and `SELECT` to use the function.
 - **MySQL Syntax Used:** `CREATE TABLE` , `INSERT INTO` , `DELIMITER` , `CREATE PROCEDURE ... (IN ...) BEGIN ... END` , `CALL ...(...)` , `CREATE FUNCTION ... (...) RETURNS ... DETERMINISTIC BEGIN DECLARE ...; SELECT ... INTO ...; RETURN ...; END` , `SELECT function_name(...)` . (Also `INT` , `VARCHAR` , `DECIMAL` , `PRIMARY KEY` , `AUTO_INCREMENT` , `NOT NULL`).
-

Experiment No. 9

- **Title:** Views and Triggers.
- **Summary:** This experiment covered two advanced database objects: Views and Triggers. Views (`CREATE VIEW`) are virtual tables based on the result set of a stored SQL query, simplifying complex queries and controlling data access. Triggers (`CREATE TRIGGER`) are stored procedures that automatically execute (fire) in response to specific DML events (like `INSERT` , `UPDATE` , `DELETE`) on a particular table.
- **Code/Implementation:** Created `Departments` and `Employees_SE` tables. Created a view `HighSalaryEmployees` showing employees with `Salary > 6000` . Created a `SalaryChanges` table to log modifications. Created an `AFTER UPDATE` trigger `AfterSalaryUpdate` on `Employees_SE` which automatically inserted the `OLD.Salary` and `NEW.Salary` into `SalaryChanges` whenever an employee's salary was updated. Demonstrated querying the view and observing the trigger's action after an `UPDATE` .
- **MySQL Syntax Used:** `CREATE TABLE` , `INSERT INTO` , `CREATE VIEW ... AS SELECT ...` , `SELECT * FROM view_name` , `CREATE TABLE ... (... , TIMESTAMP DEFAULT CURRENT_TIMESTAMP)` , `DELIMITER` , `CREATE TRIGGER ... AFTER UPDATE ON ... FOR EACH ROW BEGIN INSERT INTO ... VALUES (OLD.col, NEW.col); END` , `UPDATE ... SET ... WHERE ...` . (Also `INT` , `VARCHAR` , `DECIMAL` , `PRIMARY KEY` , `AUTO_INCREMENT` , `NOT NULL` , `FOREIGN KEY`).

Experiment No. 10

- **Title:** Mini project- Course project report using database connectivity.
- **Summary:** This was a culminating mini-project. The goal was to build a simple two-tier client-server application (e.g., using Java and JDBC, as suggested by the objective) that connects to the database designed and implemented in the previous experiments. The focus was on demonstrating the ability to interact with the database (perform CRUD operations, potentially call procedures/functions) from an external application layer.
- **Code/Implementation:** No specific SQL code provided in the lab sheet itself. The implementation required writing application code (e.g., Java) using database connectivity APIs (JDBC/ODBC) to interact with the MySQL database, followed by preparing a report and demonstrating the working application.
- **MySQL Syntax Used (by the application):** Likely involved standard DML (`SELECT` , `INSERT` , `UPDATE` , `DELETE`) and potentially `CALL` (for procedures) or `SELECT function_name()` (for functions), executed through the database connection API from the application code.

Here is a summary list of MySQL syntax:

1. Basic Syntax

- **Selecting Data**

```
SELECT column1, column2 FROM table_name WHERE condition;
```

Example:

```
SELECT first_name, last_name FROM employees WHERE department = 'Sales';
```

- **Inserting Data**

```
INSERT INTO table_name (column1, column2) VALUES (value1, value2);
```

Example:

```
INSERT INTO employees (first_name, last_name, department) VALUES ('John', 'Doe', 'Marketing');
```

- **Updating Data**

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

Example:

```
UPDATE employees SET department = 'HR' WHERE last_name = 'Doe';
```

- **Deleting Data**

```
DELETE FROM table_name WHERE condition;
```

Example:

```
DELETE FROM employees WHERE department = 'Marketing';
```

- **Creating a Table**

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
);
```

Example:

```
CREATE TABLE employees (  
    employee_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    department VARCHAR(50)  
);
```

- **Altering a Table (Adding Columns)**

```
ALTER TABLE table_name ADD column_name datatype;
```

Example:

```
ALTER TABLE employees ADD birth_date DATE;
```

- **Dropping a Table**

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE employees;
```

2. Joins

- **INNER JOIN**

```
SELECT column1, column2 FROM table1 INNER JOIN table2 ON table1.id = table2.id;
```

Example:

```
SELECT employees.first_name, departments.name  
FROM employees  
INNER JOIN departments ON employees.department_id = departments.id;
```

- **LEFT JOIN**

```
SELECT column1, column2 FROM table1 LEFT JOIN table2 ON table1.id = table2.id;
```

Example:

```
SELECT employees.first_name, departments.name  
FROM employees  
LEFT JOIN departments ON employees.department_id = departments.id;
```

- **RIGHT JOIN**

```
SELECT column1, column2 FROM table1 RIGHT JOIN table2 ON table1.id = table2.id;
```

3. Advanced Queries

- **GROUP BY**

```
SELECT column1, COUNT(column2) FROM table_name GROUP BY column1;
```

Example:

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```


- **ORDER BY**

```
SELECT column1, column2 FROM table_name ORDER BY column1 ASC|DESC;
```

Example:

```
SELECT first_name, last_name FROM employees ORDER BY last_name DESC;
```

- **DISTINCT**

```
SELECT DISTINCT column_name FROM table_name;
```

Example:

```
SELECT DISTINCT department FROM employees;
```

4. Stored Procedures

- **Creating a Stored Procedure**

```
DELIMITER //  
CREATE PROCEDURE procedure_name()  
BEGIN  
    -- SQL Statements  
END //  
DELIMITER ;
```

Example:

```
DELIMITER //  
CREATE PROCEDURE GetEmployeesByDepartment(dep VARCHAR(50))  
BEGIN  
    SELECT first_name, last_name FROM employees WHERE department = dep;  
END //  
DELIMITER ;
```

- **Calling a Stored Procedure**

```
CALL procedure_name(parameters);
```

Example:

```
CALL GetEmployeesByDepartment('HR');
```

5. Views

- **Creating a View**

```
CREATE VIEW view_name AS  
SELECT column1, column2 FROM table_name WHERE condition;
```

Example:

```
CREATE VIEW EmployeeView AS  
SELECT first_name, last_name, department FROM employees WHERE department = 'Sales';
```

- **Selecting from a View**

```
SELECT * FROM view_name;  
+
```

Example:

```
SELECT * FROM EmployeeView;
```

- **Dropping a View**

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW EmployeeView;
```

6. Triggers

- **Creating a Trigger**

```
CREATE TRIGGER trigger_name  
BEFORE|AFTER INSERT|UPDATE|DELETE ON table_name  
FOR EACH ROW  
BEGIN  
    -- Trigger Logic  
END;
```

Example:

```
CREATE TRIGGER update_employee_log  
AFTER UPDATE ON employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO employee_logs (employee_id, action)  
    VALUES (NEW.employee_id, 'Updated record');  
END;
```

- **Dropping a Trigger**


```
DROP TRIGGER trigger_name;
```

Example:

```
DROP TRIGGER update_employee_log;
```

7. Indexes

- **Creating an Index**

```
CREATE INDEX index_name ON table_name (column_name);
```

Example:

```
CREATE INDEX idx_last_name ON employees (last_name);
```

- **Dropping an Index**

```
DROP INDEX index_name ON table_name;
```

Example:

```
DROP INDEX idx_last_name ON employees;
```

8. Transactions

- **Starting a Transaction**

```
START TRANSACTION;
```

- **Committing a Transaction**

```
COMMIT;
```

- **Rolling Back a Transaction**

```
ROLLBACK;
```

9. Other Useful SQL Clauses

- **LIMIT (for Pagination)**

```
SELECT column1, column2 FROM table_name LIMIT offset, row_count;
```

Example:

```
SELECT first_name, last_name FROM employees LIMIT 10, 20;
```

- **LIKE (Pattern Matching)**

```
SELECT column1 FROM table_name WHERE column1 LIKE pattern;
```

Example:

```
SELECT first_name FROM employees WHERE first_name LIKE 'J%';
```

- **BETWEEN (Range Matching)**

```
SELECT column1 FROM table_name WHERE column1 BETWEEN value1 AND value2;
```

Example:

```
SELECT first_name FROM employees WHERE employee_id BETWEEN 10 AND 50;
```

This should give you a solid foundation of MySQL syntax from basic queries to more advanced concepts such as stored procedures, views, and triggers. Let me know if you'd like to dive deeper into any specific area!